

Description of Load Testing

Load testing is a subtype of performance testing that involves collecting metrics and determining the performance and response time of a software/hardware system or device in response to an external request. This is done to establish compliance with the requirements set for that system (or device). The term "load testing" can be used with different meanings within the professional software testing community. In general, it refers to the practice of simulating expected application usage by emulating the work of multiple users simultaneously. Therefore, this type of testing is most suitable for multi-user systems, often those using a client-server architecture (e.g., web servers). However, other types of software systems can also be tested in a similar way. For example, a text or graphics editor can be forced to read a very large document, or a financial package can be made to generate a report based on data from several years. A properly designed load test yields the most accurate results.

The main goal of load testing is to create a certain expected load on the system (e.g., via virtual users) and, typically, using identical software and hardware, to observe the system's performance metrics.

Some tools for load testing are listed below:

Software	Vendor/Developer	Description
OpenSTA	Open System Testing Architecture	Free and open-source software for load/stress testing, licensed under GNU GPL. Uses a distributed application architecture based on CORBA. A Windows version is available, though it has compatibility issues with Windows Vista. Support was discontinued in 2007.
IBM Rational Performance Tester	IBM	Eclipse-based software that enables generating high-volume loads and measuring response times for client-server applications. Requires licensing.
JMeter	Apache Jakarta	Java-based cross-platform toolkit for

Software	Vendor/Developer	Description
	Project (Open Source)	conducting load tests using JDBC, FTP, LDAP, SOAP, JMS, POP3, HTTP, TCP connections. Allows generating a large number of requests from multiple computers and controlling the process from a single machine.
HP LoadRunner	HP	A load testing tool originally designed to emulate the work of a large number of concurrent users. Can also be used for unit or integration testing.
LoadComplete	SmartBear	Proprietary product for conducting load testing of web applications.
SilkPerformer	Micro Focus (Borland)	
Siege	Joe Dog Software	Siege is a utility for load testing web servers.
Visual Studio Team System	Microsoft	Visual Studio provides a performance testing tool, including load and unit testing.
QTest	Quotium	
HTTPerf		
QALoad	Compuware Ltd.	
(The) Grinder		

Software	Vendor/Developer	Description
WebLOAD	RadView Software	A load testing tool for web and mobile applications, including web dashboards for performance test analysis. Used for large-scale loads that can also be generated from the cloud. Licensed software.
Yandex.Tank	Yandex	A modular and extensible tool that allows using various internal load generators, notably the widely-known JMeter. This is an open-source project published by Yandex in 2012.

Key Performance Metrics

One of the results obtained during load testing and used for subsequent analysis is the application's performance metrics. The main ones are detailed below.

1. CPU Utilization, %

This metric shows the percentage of a given time interval the processor spent on calculations for a selected process. In modern systems, a process's ability to work in multiple threads is a crucial factor, enabling the processor to perform computations in parallel. Analysis of CPU usage history can explain the impact on overall system performance of factors such as processed data threads, application and operating system configuration, multithreaded computations, and others.

2. Memory (RAM) Consumption, MB

This metric shows the amount of memory used by an application. The used memory is divided into several categories:

Virtual: The amount of virtual address space used by a process. This amount implies the use of both corresponding disk space and RAM. The virtual memory system ensures that threads of one process cannot access memory belonging to another process.

Private: The amount of address space occupied by a process and not shared with other processes.

Working Set: The set of memory pages recently used by a process. When free memory is sufficient, pages remain in the set even if they are not in use. When free memory becomes scarce, used pages are moved from RAM to the hard disk (or another storage device, such as flash memory), freeing up RAM for loading other active memory pages.

Shared: The amount of physical memory used by a process that can be shared with other processes. Although memory allocated to a process should be isolated, processes sometimes need to be able to exchange information. Shared memory is the fastest method of inter-process communication.

During application operation, memory is filled with references to objects which, if unused, can be cleaned up by a special automatic process called garbage collection. The time the processor spends cleaning memory in this way can be significant when the process has occupied all available memory (in Java — the so-called "persistent Full GC") or when the process has been allocated large volumes of memory needing cleanup. During the time required for memory cleanup, the process's access to the pages of allocated memory may be blocked, which can affect the final time this process takes to process data.

3. Network Resource Consumption

This metric is not directly related to application performance; however, its indicators can point to the performance limits of the system as a whole.

4. Disk Subsystem Activity (I/O Wait Time)

Interaction with the disk subsystem can significantly affect system performance, so collecting statistics on disk activity can help identify bottlenecks in this area. A large number of read or write operations can lead to the processor idling while waiting for data to be processed from the disk, ultimately increasing CPU consumption and response time.

5. Request Execution Time, ms

The time an application takes to execute a request remains one of the most important indicators of system or application performance. This time can be measured on the server side as an indicator of the time required by the server component to process a request; and on the client side as an indicator of the total time required for serialization/deserialization, transmission, and processing of the request.

It should be noted that not every performance testing application can measure both of these times.

Load Testing Tool Apache JMeter

JMeter is a very powerful load testing tool capable of generating a large number of simultaneous requests through parallel operation across multiple computers. It supports plugins, allowing for extended functionality. Although JMeter was initially developed as a tool for testing web applications, it is now capable of conducting load tests for JDBC connections, FTP, LDAP, SOAP, JMS, POP3, IMAP, HTTP, and TCP. The program implements virtual user authorization mechanisms and supports user sessions. It provides logging of test results and diverse result visualization in the form of charts, tables, etc.

Installing Apache JMeter

Apache JMeter can be downloaded via the following link (the program requires Java 8+): https://jmeter.apache.org/download_jmeter.cgi

Apache JMeter 5.2 (Requires Java 8+)

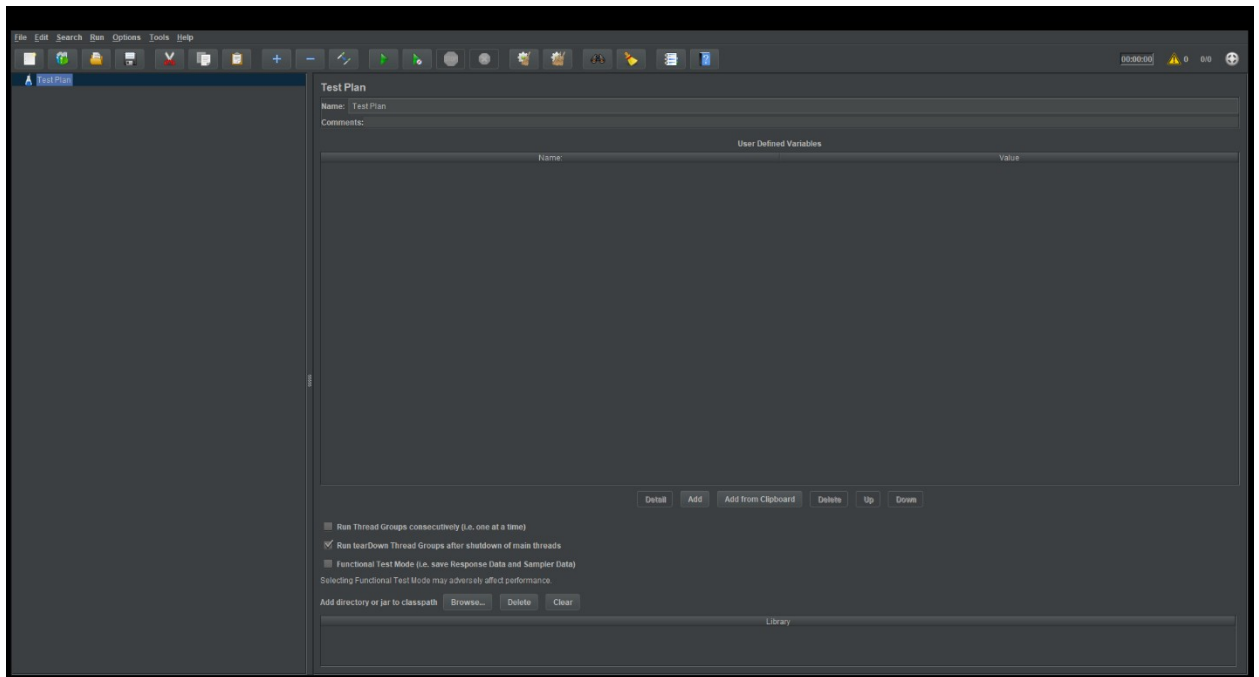
Binaries

[apache-jmeter-5.2.tgz sha512 pgp](#)
[apache-jmeter-5.2.zip sha512 pgp](#)

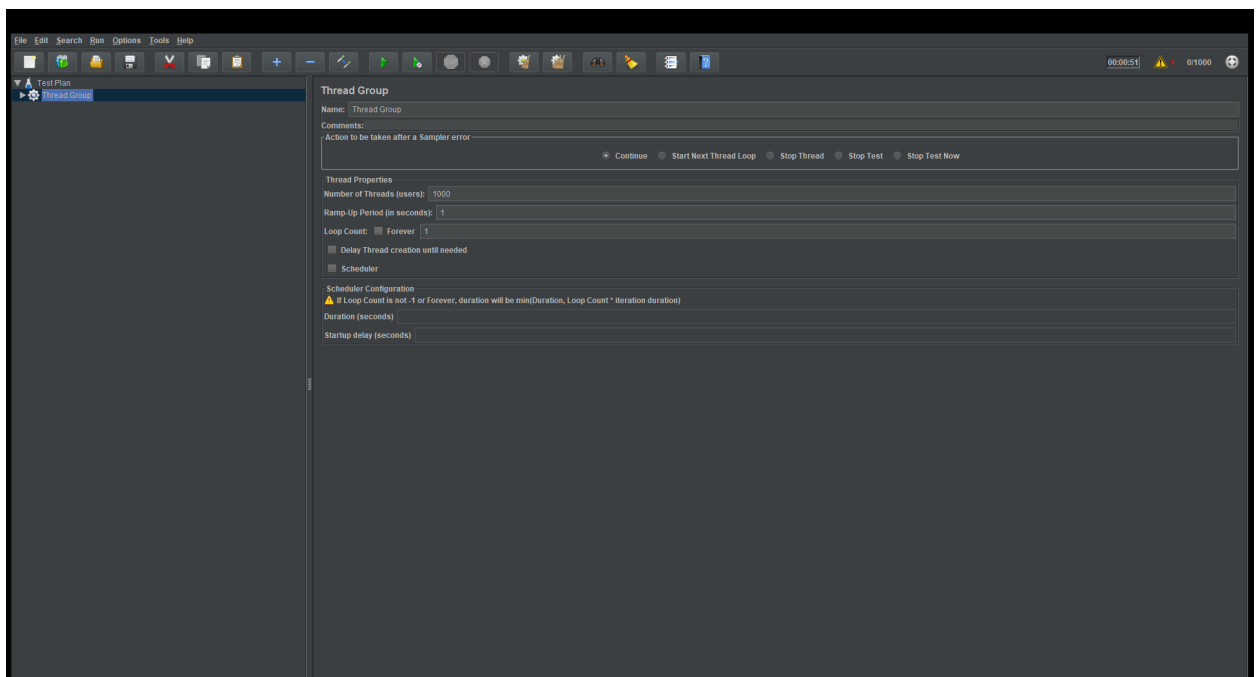
Example of Load Testing Using Apache JMeter

To demonstrate the functionality of the Apache JMeter tool, the website <https://jsonplaceholder.typicode.com/> was selected. This service allows sending various web requests. By analyzing the server's response time to requests—a key performance indicator—conclusions can be drawn about the load the tested system can handle.

Navigate to the JMeter folder, then to the bin folder, and launch ApacheJMeter.jar using the Java Virtual Machine. The following window *will open*:

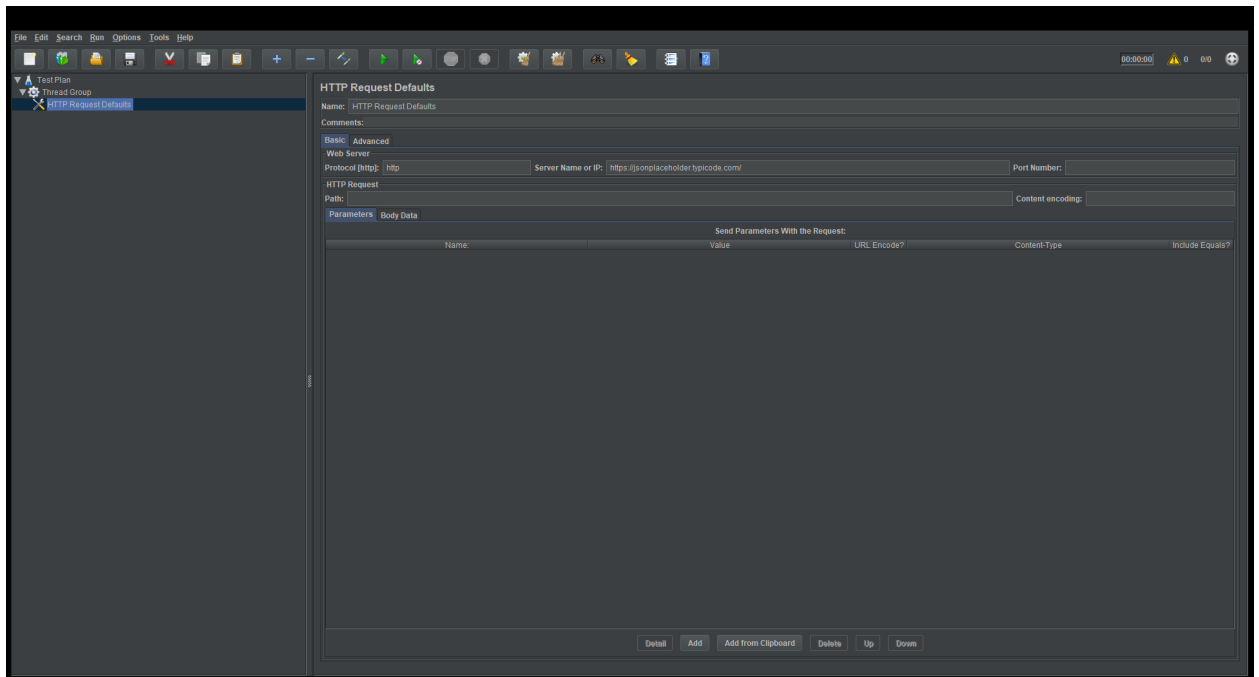


2. Right-click on the **Test Plan** element => **Add** => **Threads (Users)** => **Thread Group**.
3. In the **Number of Threads (users)** field (this is the number of requests of each type that will be added), enter the value **"1000"**. In the **Ramp-Up Period (seconds)** field (the period over which these requests will be sent) – **"1"**. In the **Loop Count** field (this field specifies how many times the request will be executed) – **"1"**.

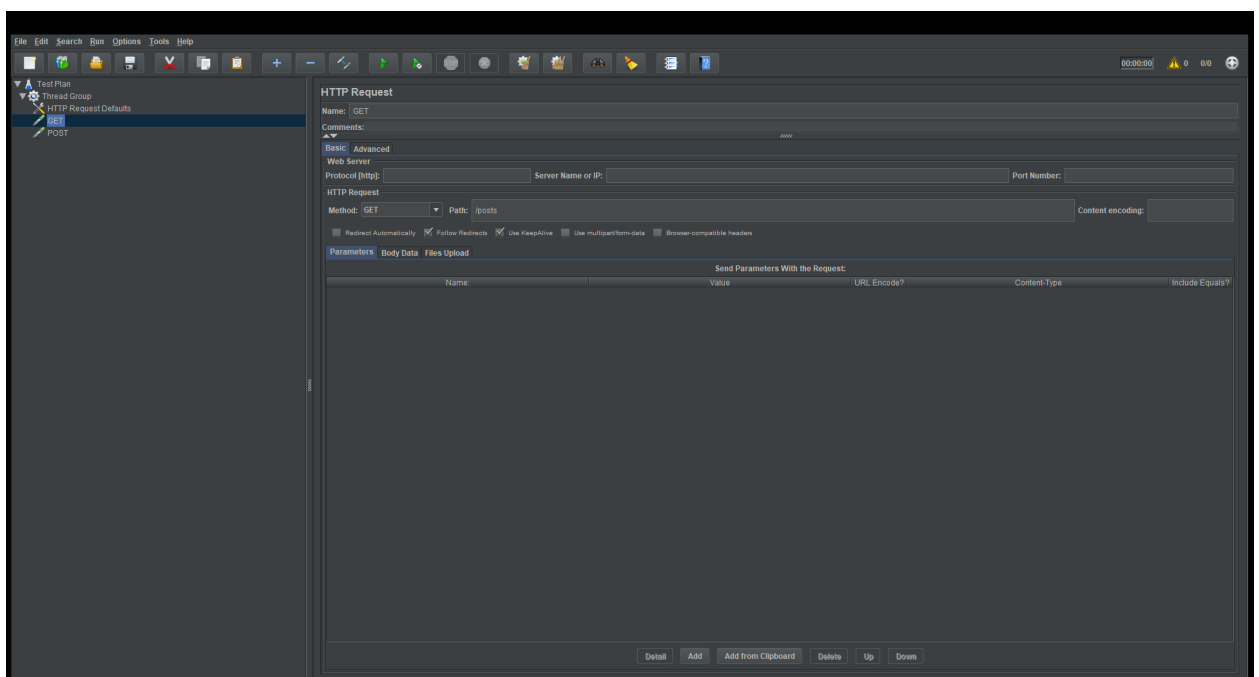


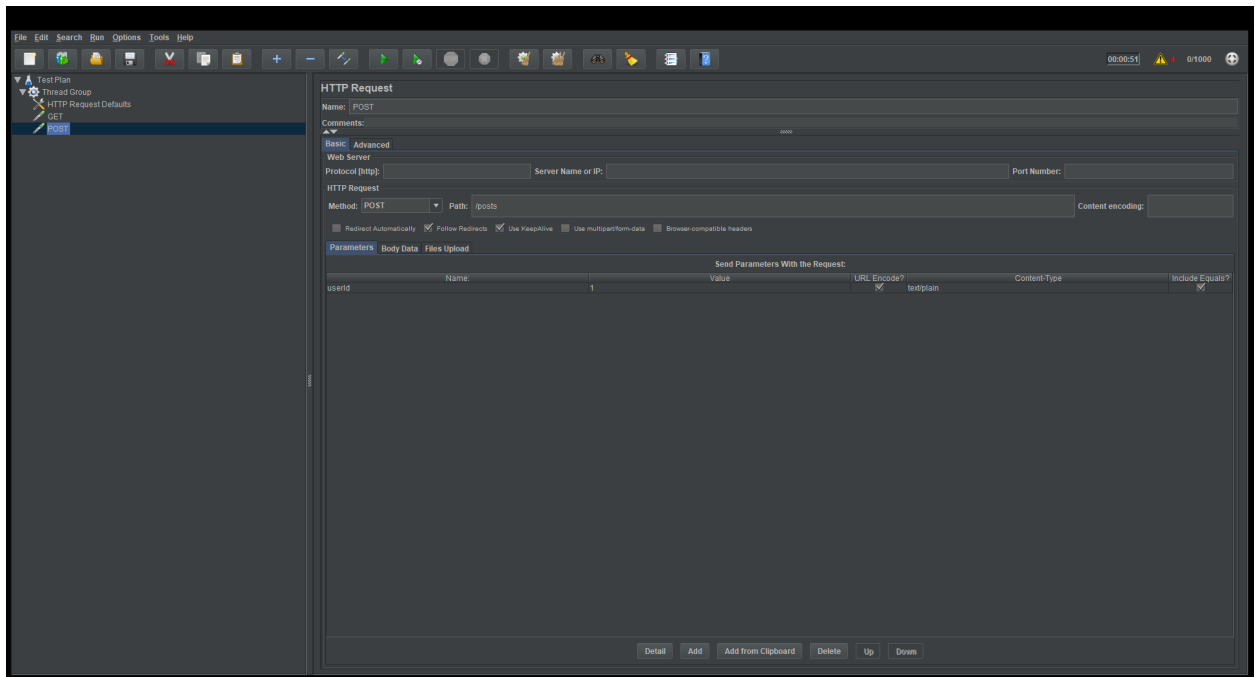
4. Right-click on the **Thread Group** element => **Add** => **Config Element** => **HTTP Request Defaults** to add the settings for the website we will be testing.

5. In the **Protocol** field, enter "**http**", and in **Server Name or IP** – "**jsonplaceholder.typicode.com**":



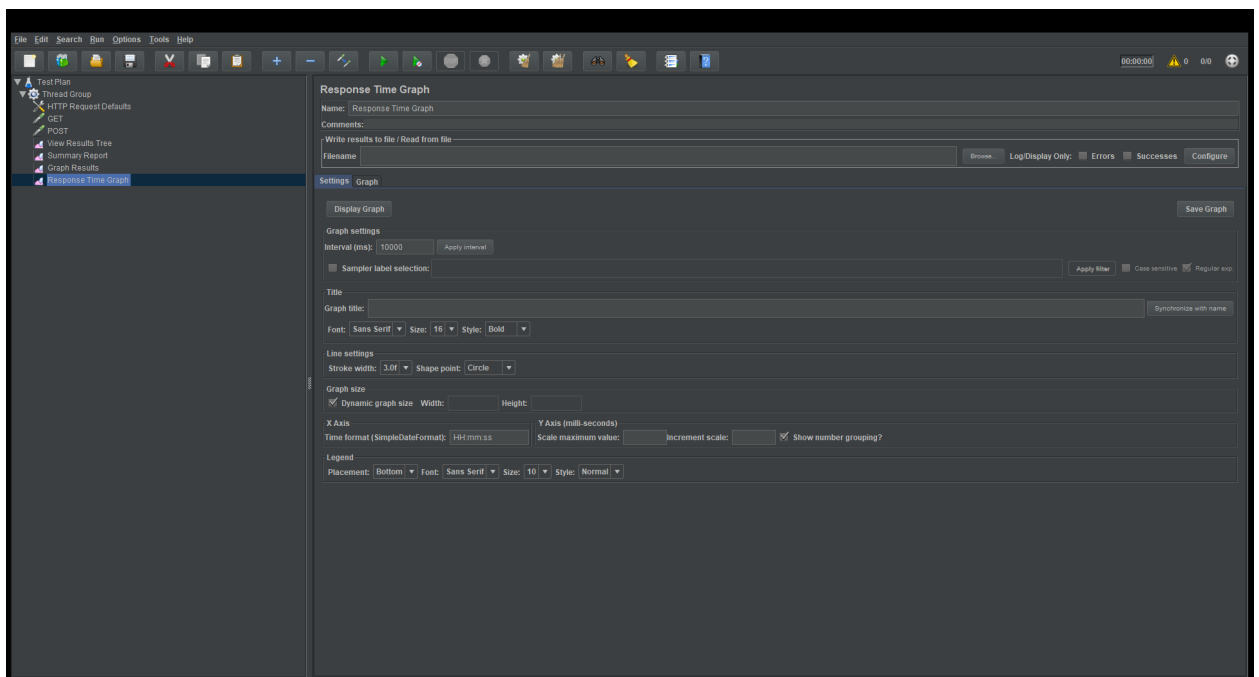
6. Add two requests, GET and POST. To do this, right-click on the **Thread Group** element => **Add** => **Sampler** => **HTTP Request**.
7. For the GET request, select "**GET**" in the **Method** field. For the POST request, select "**POST**" in the **Method** field. In the **Path** field for both requests, enter the value "**/posts**".
For the POST request, click the **Add** button with the left mouse button. In the **Name** field, add the value "**userId**", and in the **Value** field – "**1**".



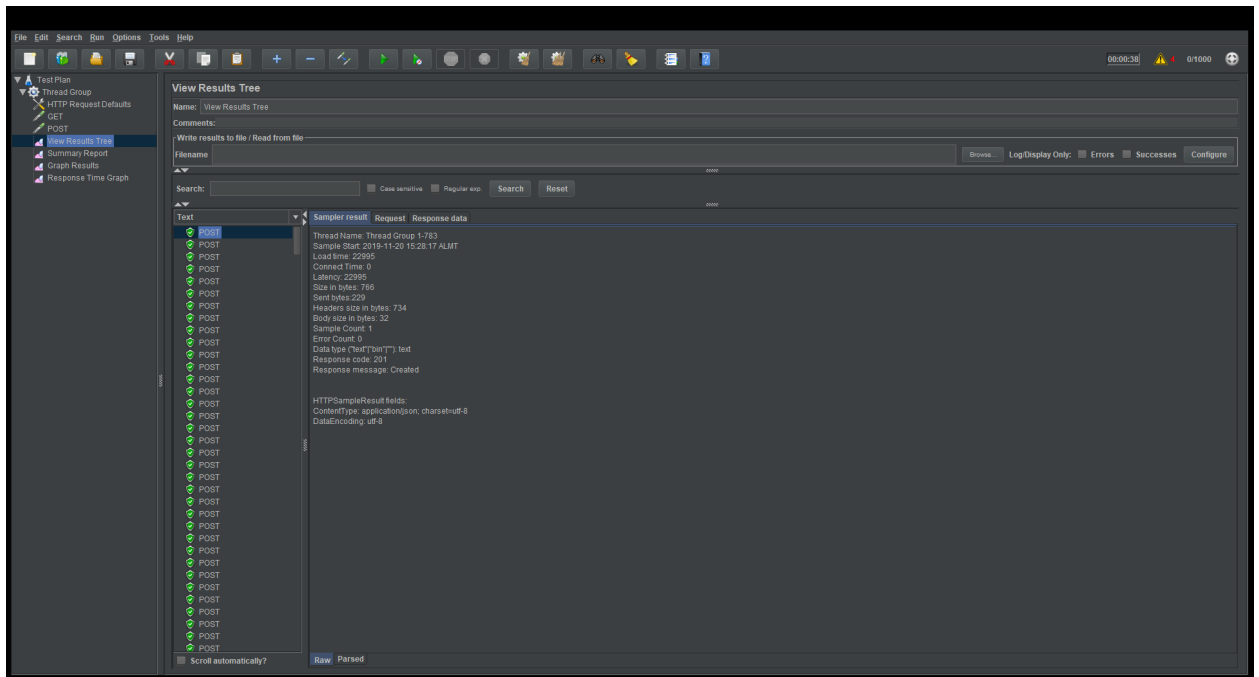


Each GET and POST request will be executed 1000 times, as defined in step 3.

8. To view the test results, right-click on the **Thread Group** element and add the following listeners: **View Results Tree**, **Summary Report**, **Graph Results**, **Response Time Graph** (via **Add => Listener**).



9. To send the requests, left-click on the **Run (Start)** button.



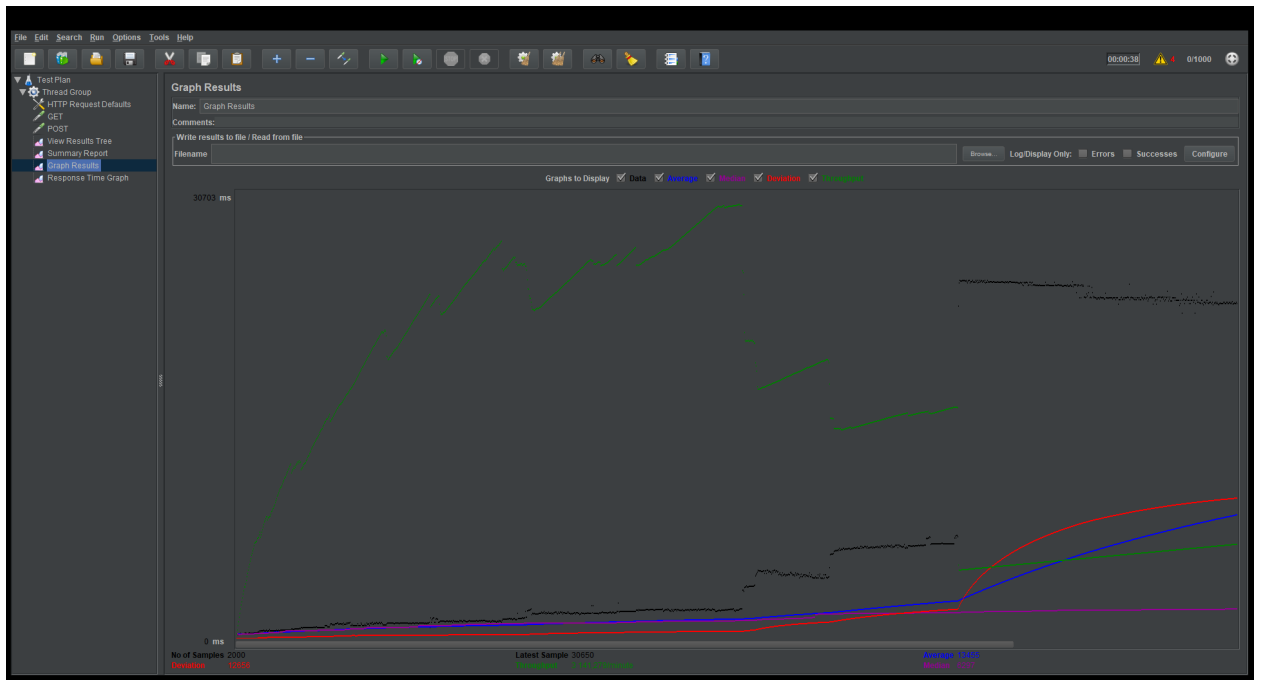
View Results Tree displays what the server responded to each specific request and how many requests were completed successfully.

The screenshot shows the 'Summary Report' panel in JMeter. It displays a table with performance metrics for GET and POST requests. The table includes columns for Label, # Samples, Average, Min, Max, Std. Dev., Error %, Throughput, Received KBits/sec, Sent KBits/sec, and Avg. Bytes. The data is as follows:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KBits/sec	Sent KBits/sec	Avg. Bytes
GET	1000	2413	289	7091	2051.49	0.00%	131.7/sec	3621.71	17.62	28163.3
POST	1000	2466	780	35463	3204.48	37.90%	26.4/sec	21.22	5.91	321.4
TOTAL	2000	13455	289	35463	12655.11	18.95%	52.4/sec	740.97	9.38	14492.5

Summary Report displays data for each type of request. Values are provided in milliseconds.

- **Data:** The response time for each individual data unit, i.e., each tested URL.
- **Average:** The average response time, an objective indicator of load changes.
- **Median:** The median value (used in statistics; I do not use this data).
- **Deviation:** Error, standard deviation.
- **Throughput:** The throughput of executed requests.



Graph Results displays the data from the Summary Report in a graphical form.